repositories. Now, with contract testing, teams can work in parallel when a contract is defined up front. Again, communication, as contract testing, helps facilitate intricate conversations about contract versions and backward compatibility.

3.6 Contract testing tools

Tooling is always a difficult decision in software, and deciding which tool is best for your context, the best documentation, and the shallowest learning curve are all key points to consider.

The beauty of contract testing is that the proposed process can be engineered with limited tooling. A custom solution was created to enable the contract testing life cycle in one of the author’s projects. The architecture involved integrating with Oracle database processes, and the tricky part of implementing contract testing within this solution was that we were trying to confirm that the multiple processes were finished successfully. The solution can be explained in the following steps and also shown in figure 3.9:

1. A web application, which is the consumer, triggers a message that gets put on a queue.
2. The message content published on the queue acts as a contract and is stored in a GitHub repository to share with the database service team.
3. The database team posts the message contract onto their queue from the provider side, triggering the database processes.
4. Finally, the database is polled with a Status column indicating whether the message passed successfully.